

BÁO CÁO CUỐI KỲ HỆ THỐNG GIÁM SÁT GIAO THÔNG THÔNG MINH VÀ NHẬN DIỆN BIỂN SỐ XE VIỆT NAM

Nguyễn Hải Đăng - 22001560

Mai Hoàng Anh - 22001538

GVHD: TS Nguyễn Thị Bích Thủy

Môn học: MAT3377 – Một số vấn đề chọn lọc về Trí tuệ nhân tạo

Nội dung trình bày

- ① Tổng quan
- ② Cơ sở lý thuyết
- ③ Phương pháp triển khai
- ④ Kết quả thực nghiệm
- ⑤ Demo và Kết luận

Bối cảnh và Nhu cầu

Bối cảnh

- Đô thị hóa tăng nhanh tại các thành phố lớn
- Mật độ phương tiện cao, chủ yếu xe máy
- Vi phạm phổ biến: vượt đèn đỏ, đi sai làn

Nhu cầu

- Giám sát tự động, độ chính xác cao
- Phù hợp giao thông Việt Nam
- Chi phí triển khai thấp

Thách thức

Mật độ cao, ánh sáng thay đổi, đối tượng bị che khuất, biển số đa dạng

Mục tiêu đề tài

Mục tiêu chính

Xây dựng hệ thống giám sát giao thông thông minh với các chức năng:

- **Phát hiện** đa loại phương tiện (5 lớp)
- **Theo dõi** quỹ đạo đối tượng liên tục
- **Nhận diện biển số** xe Việt Nam (ALPR)
- **Phát hiện vi phạm** giao thông tự động

Yêu cầu kỹ thuật

- Xử lý thời gian thực (Real-time) trên GPU phổ thông
- Độ chính xác cao
- Giao diện trực quan, dễ cấu hình

Kiến trúc tổng thể hệ thống

Luồng xử lý (Data Pipeline)

Video Input → **Detection** → **Tracking** → **ALPR** → **Violation Check**

4 Module chính:

- 1 **Vehicle Detection:** YOLOv8n (5 lớp phương tiện)
- 2 **Multi-Object Tracking:** ByteTrack
- 3 **License Plate Recognition:** YOLO + PaddleOCR
- 4 **Violation Detection:** Logic Engine (vượt đèn đỏ, sai làn)

Đặc điểm:

- Đa luồng trên PyQt5 (QThread)
- Hỗ trợ cấu hình ROI, làn đường, vạch dừng
- Xuất kết quả: Video, CSV, ảnh vi phạm

Hai phương pháp ALPR (Pipeline)

Pipeline 1: Two-Stage

- **Bước 1:** YOLOv8n phát hiện xe
- **Bước 2:** YOLOv8n phát hiện biển số
- **Bước 3:** PaddleOCR nhận dạng ký tự

Pipeline 2: One-Stage

- **Bước 1:** YOLOv8n phát hiện 6 lớp (xe + biển số)
- **Bước 2:** PaddleOCR nhận dạng ký tự

Tổng quan về Phát hiện đối tượng

- **Định nghĩa:** Xác định vị trí và phân loại các đối tượng trong ảnh/video
- **Đầu ra:** Tập hợp bounding box $B = \{b_1, b_2, \dots, b_N\}$

$$b_i = (x_i, y_i, w_i, h_i, c_i, p_i)$$

- (x_i, y_i) : Tọa độ tâm
 - (w_i, h_i) : Kích thước khung bao
 - c_i : Nhãn lớp (xe máy, ô tô, ...)
 - p_i : Độ tin cậy (confidence)
- **Thách thức:** Biến thiên tỷ lệ, bị che khuất, biến dạng hình học

Lựa chọn mô hình: YOLOv8

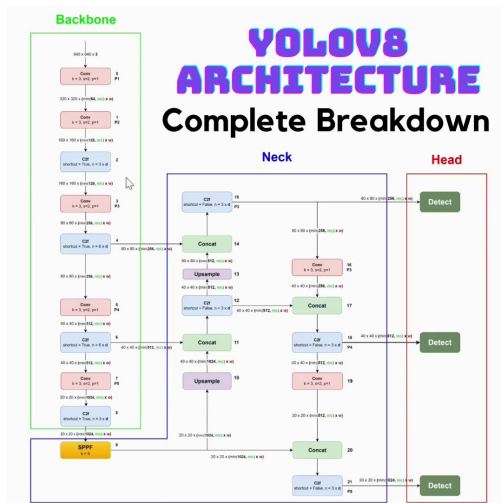
Lý do lựa chọn YOLOv8 cho bài toán giao thông:

- **Anchor-free:** Loại bỏ anchor box thủ công, linh hoạt với các đối tượng có tỷ lệ biến thiên lớn
- **Module C2f:** Cải thiện luồng gradient, trích xuất đặc trưng phong phú
- **Task Aligned Assigner:** Cân bằng giữa điểm phân loại và độ chính xác định vị
- **Hiệu năng:** Cân bằng tốt giữa tốc độ (Real-time) và độ chính xác (mAP)

YOLOv8n (Nano)

Phiên bản nhẹ nhất (3.2M params), phù hợp cho ứng dụng real-time trên GPU phổ thông

Kiến trúc YOLOv8



Hình: Kiến trúc tổng thể YOLOv8

Hàm mất mát YOLOv8

Tối thiểu hóa hàm mất mát tổng hợp:

$$L_{total} = \lambda_{box} L_{box} + \lambda_{cls} L_{cls} + \lambda_{dfl} L_{dfl}$$

Các thành phần chính

- **CloU Loss** (L_{box}): Định vị bounding box

$$\mathcal{L}_{CloU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$$

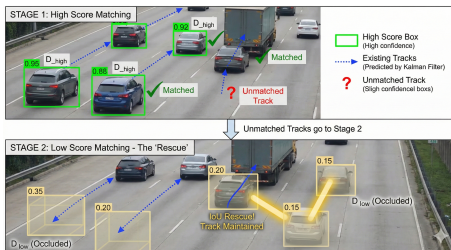
- **Distribution Focal Loss (DFL)**: Tối ưu cho box có biên mờ
- **Varifocal Loss** (L_{cls}): Cân bằng mẫu khó/dễ

Theo dõi đa đối tượng (MOT)

Mục tiêu: Gán ID duy nhất cho mỗi đối tượng qua các khung hình liên tiếp

Phương pháp: Tracking-by-Detection

- **Bộ lọc Kalman:** Ước lượng trạng thái (vị trí, vận tốc) để dự đoán vị trí tiếp theo
- **Thuật toán Hungarian:** Ghép cặp tối ưu giữa *Dự đoán Kalman* và *Phát hiện YOLO*



Ý tưởng: "Tận dụng mọi detection, kể cả độ tin cậy thấp"

Quy trình 2 giai đoạn

① High Score Matching:

- Ghép detection chất lượng cao ($conf > \tau_{high}$) với Track hiện có
- Sử dụng IoU làm trọng số

② Low Score Matching:

- Track chưa ghép sẽ tìm trong detection chất lượng thấp ($conf > \tau_{low}$)
- "Cứu" lại xe bị che khuất hoặc mờ

Ưu điểm: Tốc độ cao (không cần trích xuất đặc trưng ngoại hình)

Nhận diện đèn giao thông (HSV)

Chuyển đổi RGB $\rightarrow$ HSV

Tách biệt thông tin màu sắc và độ sáng

Ngưỡng màu cho từng trạng thái đèn:

- **Đỏ (Red):** $Hue \in [0, 10] \cup [160, 180]$ (red-wrap effect)
- **Vàng (Yellow):** $Hue \in [15, 35]$
- **Xanh (Green):** $Hue \in [40, 90]$

Cơ chế Voting

Tính tỷ lệ pixel cho từng mặt nạ màu $\rightarrow$ Xác định trạng thái trội

Hỗ trợ 4 loại đèn: *Tròn, Rẽ trái, Rẽ phải, Đi thẳng*

Thuật toán phát hiện Vượt đèn đỏ

Quy trình quyết định (Decision Flow)

Khi sự kiện *Stopline Crossing* xảy ra:

① Kiểm tra Quyền đi (Green Priority):

- Có đèn xanh mũi tên trùng hướng xe? → **HỢP LỆ**

② Kiểm tra Cấm (Red Constraint):

- Đèn đỏ tròn hoạt động? HOẶC Đèn đỏ mũi tên trùng hướng xe?
- → Nếu Đúng: **VI PHẠM**

③ Xác nhận: Loại trừ ngoại lệ (xe ưu tiên)

Thuật toán phát hiện Đi sai làn

Cấu hình Lane Manager:

- Mỗi làn là một *Polygon* + danh sách *allowed_classes*
- Hỗ trợ làn đè (overlapping lanes)

Quy trình kiểm tra

- 1 Tính tọa độ tâm (C_x, C_y) của xe từ Bounding Box
- 2 **Point-in-Polygon Test:** Xác định xe thuộc làn L_i nào
- 3 **Class Verification:**

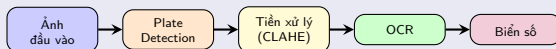
$$\text{Violation} = (C \in L_i) \wedge (Class_v \notin L_i.allowed)$$

Sử dụng *Ray Casting Algorithm* cho *Point-in-Polygon*

Tổng quan ALPR (Nhận diện biển số)

ALPR = Automatic License Plate Recognition

Pipeline tổng quát



Thách thức nhận diện biển số Việt Nam:

- Đa dạng định dạng: 1 dòng, 2 dòng
- Điều kiện thực tế: Bắn, mờ, nghiêng
- Ánh sáng: Phản chiếu, bóng đổ

Biển số xe Việt Nam

Loại	Định dạng	Ví dụ
Xe ô tô (1 dòng)	XX-Y XXXXX	30G-07784
Xe ô tô (2 dòng)	XX-Y / XXXXX	30G / 07784
Xe máy	XX-YZ / XXX.XX	29-H1 / 234.56

Cấu trúc:

- **2 số đầu:** Mã tỉnh (30 = Hà Nội, 51 = TP.HCM)
- **1 chữ cái:** Seri (A-Z, trừ I, O, Q)
- **5 số cuối:** Số thứ tự đăng ký

CRNN = Convolutional Recurrent Neural Network

3 thành phần chính

- ① **CNN Backbone:** Trích xuất đặc trưng không gian từ ảnh
 - Sử dụng MobileNet/ResNet biến thể
- ② **Sequence Modeling (Bi-LSTM):**
 - Mô hình hóa quan hệ tuần tự giữa các ký tự
 - Bidirectional: capture thông tin cả 2 hướng
- ③ **CTC Decoder:** Giải mã chuỗi ký tự cuối cùng

Ảnh biển số → CNN → Bi-LSTM → CTC → “30G07784”

CTC Loss (Connectionist Temporal Classification)

Đặc điểm: Huấn luyện không cần alignment giữa input và output

Công thức

Xác suất của chuỗi output l :

$$P(l|x) = \sum_{\pi \in \mathcal{B}^{-1}(l)} P(\pi|x)$$

CTC Loss = Negative log-likelihood:

$$\mathcal{L}_{CTC} = -\log P(l|x)$$

Ví dụ: Chuỗi mục tiêu “AB”

- “A-A-B-B” $\rightarrow$ “AB” (loại bỏ ký tự lặp)
- “A-B-” $\rightarrow$ “AB” (loại bỏ blank ‘-’)

CLAHE = Contrast Limited Adaptive Histogram Equalization

Nguyên lý hoạt động

- 1 **Chia tile:** Chia ảnh thành các tile nhỏ (8×8 pixels)
- 2 **HE cục bộ:** Histogram Equalization trên từng tile
- 3 **Clip Limit:** Giới hạn độ tương phản (tránh khuếch đại noise)

$$\beta = \frac{M \times N}{L} \times \text{clipLimit}$$

- 4 **Bilinear Interpolation:** Ghép các tile mượt mà

Cấu hình: clipLimit=2.0, tileGridSize=(8,8)

Sharpening - Làm sắc nét

Mục đích: Tăng cường cạnh ký tự trước khi đưa vào OCR

Kernel Sharpening

$$K_{sharpen} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Công thức pixel mới:

$$I'(x, y) = 5 \cdot I(x, y) - [I(x - 1, y) + I(x + 1, y) + I(x, y - 1) + I(x, y + 1)]$$

Tăng cường sự khác biệt giữa pixel và các pixel xung quanh

→ *Cạnh sắc nét hơn*

Character Correction - Hiệu chỉnh ký tự

Vấn đề: OCR có thể nhầm lẫn ký tự tương tự

Sai	Đúng	Điều kiện
0 (số)	O (chữ)	Vị trí chữ cái (index 2)
O (chữ)	0 (số)	Vị trí chữ số
1 (số)	I (chữ)	Vị trí chữ cái
8 (số)	B (chữ)	Vị trí chữ cái

Logic hiệu chỉnh theo định dạng VN:

- Vị trí 0-1: Bắt buộc chữ số (mã tỉnh)
- Vị trí 2: Bắt buộc chữ cái (seri)
- Vị trí 3+: Bắt buộc chữ số

Format Validation & Retry Mechanism

Vấn đề: OCR có thể cho kết quả khác nhau giữa các frame

- Biển số bị rung, blur khi xe di chuyển
- Ánh sáng thay đổi, góc nhìn thay đổi

Giải pháp: Format Validation

Validate theo định dạng biển số Việt Nam:

- **Xe máy:** [0-9]{2}[A-Z][A-Z0-9][0-9]{4,5} (8-9 ký tự)
- **Ô tô:** [0-9]{2}[A-Z][0-9]{4,5} (7-8 ký tự)

Retry Mechanism:

- Sai format → không lưu, thử lại frame sau
- Đã có kết quả valid → dừng OCR (tiết kiệm tài nguyên)

Xác định hướng di chuyển

Direction Fusion - Kết hợp 2 nguồn:

Nguồn 1: Trajectory

- Tính vector chuyển động từ lịch sử vị trí
- So sánh với Reference Vector
- Khử nghiêng camera

Nguồn 2: ROI-based

- Xác định hướng dựa trên vùng xe đang đi qua
- Cấu hình sẵn cho từng làn

Góc lệch:

$$\alpha = \text{atan2}(\vec{v} \times \vec{r}, \vec{v} \cdot \vec{r})$$

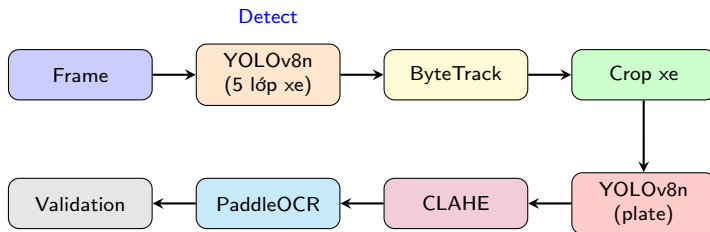
Trong đó $\vec{v}$: vector chuyển động xe, $\vec{r}$: vector tham chiếu đường

Tóm tắt các mô hình sử dụng

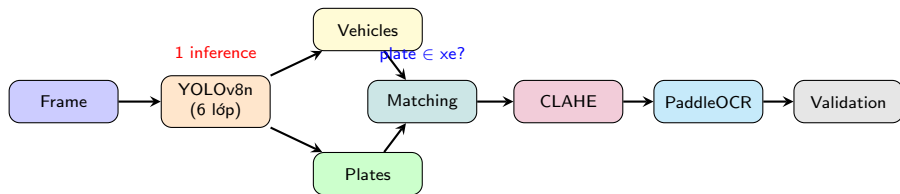
Thành phần	Mô hình	Vai trò
Phát hiện xe	YOLOv8n	5 lớp phương tiện
Phát hiện biển số	YOLOv8n	1 lớp (plate)
Mô hình tích hợp	YOLOv8n	6 lớp (5 xe + plate)
Tracking	ByteTrack	Theo dõi đa đối tượng
OCR	PaddleOCR + CLAHE	Nhận dạng ký tự biển số

Bảng: Tổng hợp các mô hình trong hệ thống

Pipeline 1: Sơ đồ triển khai (Two-Stage)



Pipeline 2: Sơ đồ triển khai (One-Stage)



Pipeline ALPR chi tiết

Quy trình nhận diện biển số (Pipeline 1)

- ① **Plate Detection:** YOLOv8n phát hiện vùng biển số
- ② **Tiền xử lý ảnh:**
 - CLAHE (Contrast Limited Adaptive Histogram Equalization)
 - Grayscale conversion
 - Noise reduction
- ③ **OCR:** PaddleOCR nhận dạng ký tự
- ④ **Hậu xử lý:**
 - Character correction ($0 \leftrightarrow O$, $1 \leftrightarrow l$)
 - Format validation (biển số Việt Nam)

Cấu hình huấn luyện

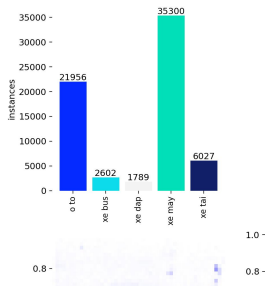
Tham số	Model 1	Model 2	Tích hợp
Base Model	YOLOv8n	YOLOv8n	YOLOv8n
Số lớp	5 (xe)	1 (plate)	6
Image Size	416×416	416×416	640×640
Batch Size	16	16	64
Epochs	100	100	100
Optimizer	SGD	AdamW	SGD

Bảng: Cấu hình huấn luyện 3 mô hình

Dữ liệu huấn luyện

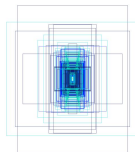
Dataset phát hiện xe:

- Nguồn: CCTV + Camera thực tế
- Quy mô: >16,000 ảnh
- Labels: >65,000 boxes
- 5 lớp: ô tô, xe máy, xe buýt, xe tải, xe đạp



Dataset biển số (VNLP):

- Nguồn: Thu thập thực tế
- Đa dạng: Nhiều góc độ, ánh sáng
- Biển số Việt Nam: 1 hàng, 2 hàng
- Fine-tuned từ pretrained weights



Pipeline 2: Mô hình tích hợp 6 lớp

Ý tưởng: Huấn luyện 1 model phát hiện đồng thời xe và biển số

- 1 Frame $\rightarrow$ YOLOv8n (6 lớp) $\rightarrow$ {Vehicles, Plates}
- 2 **Matching:** Kiểm tra Plate nằm trong Vehicle (Containment)
- 3 Crop biển số $\rightarrow$ CLAHE $\rightarrow$ PaddleOCR
- 4 Format Validation + Retry

Ưu điểm so với Pipeline 1:

- Chỉ 1 lần forward pass qua YOLO (nhANH hơn 2–3x)
- Mapping plate $\rightarrow$ xe tự động qua containment check
- Tiết kiệm GPU Memory (1.2GB vs 2.5GB)
- Model học được mối quan hệ không gian xe–biển số

Tạo dữ liệu cho Pipeline 2: Vấn đề

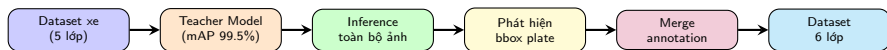
Vấn đề

- Dataset phương tiện (vietnam_vehicle_dataset): **chỉ có 5 lớp xe**
- Dataset biển số (VNLP): **chỉ có 1 lớp plate**
- **Cần:** Dataset 6 lớp (5 xe + plate) để huấn luyện model tích hợp

Giải pháp: Semi-supervised Labeling

Sử dụng Model 2 (đã train trên VNLP, mAP 99.5%) làm **Teacher Model** để tự động gán nhãn biển số cho dataset xe

Tạo dữ liệu Pipeline 2: Quy trình chi tiết



Các bước chi tiết

- 1 **Chuẩn bị:** Dataset xe (16,000+ ảnh, 5 lớp) + Teacher Model (YOLOv8n)
- 2 **Inference:** Chạy Teacher trên từng ảnh → Phát hiện bbox biển số
- 3 **Lọc:** Chỉ giữ bbox có confidence > 0.5
- 4 **Merge:** Thêm bbox plate (`class_id = 5`) vào file annotation gốc
- 5 **Kết quả:** Dataset 6 lớp (5 xe + 1 plate) sẵn sàng huấn luyện

Tạo dữ liệu Pipeline 2

Lưu ý

- Chỉ thêm bbox có confidence > 0.5
- Không xóa nhãn xe gốc (5 lớp)
- Kết quả: 65,000 boxes xe + X boxes plate

Pipeline 2: Trade-off sau khi huấn luyện

Metric	Model 2 (VNLP)	Model tích hợp
Số lớp	1 (plate)	6 (5 xe + plate)
mAP@50 (plate)	99.5%	59%
Tốc độ (FPS)	–	15–20 FPS
GPU Memory	0.8GB	1.2GB

Nguyên nhân mAP plate giảm:

- **Class imbalance:** Số lượng xe » số lượng biển số
- **Kích thước nhỏ:** Biển số chiếm diện tích rất nhỏ trong ảnh
- **Annotation noise:** Gán nhãn tự động có thể bỏ sót biển mờ/nhỏ

Pipeline 2: Gán nhãn bán giám sát (Semi-supervised)

Vấn đề: Dataset xe (5 lớp) không có nhãn biển số

Quy trình gán nhãn tự động

- 1 **Teacher Model:** Dùng Model 2 (mAP 99.5%) đã huấn luyện
- 2 **Inference:** Chạy Teacher trên toàn bộ ảnh dataset xe
- 3 **Merge:** Thêm bbox biển số vào annotation gốc
- 4 **Kết quả:** Dataset 6 lớp (5 xe + 1 plate)

Trade-off

mAP plate giảm từ 99.5% xuống 59% do:

- Class imbalance (biển số ít hơn nhiều so với xe)
- Kích thước biển số nhỏ trong image size 416px

Tiền xử lý ảnh biển số

Bước 1: CLAHE

- Cải thiện độ tương phản
- clipLimit = 2.0
- tileGridSize = (8, 8)
- Hiệu quả với ảnh tối/sáng không đều

Bước 2: Sharpening

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Làm sắc nét cạnh ký tự

Character Correction

Hiệu chỉnh theo vị trí trong biển số:

- Vị trí 0-1: Chữ số (mã tỉnh) $\rightarrow O \rightarrow 0$
- Vị trí 2: Chữ cái (seri) $\rightarrow 0 \rightarrow O$
- Vị trí 3+: Chữ số $\rightarrow B \rightarrow 8$

Định dạng biển số hợp lệ

- **Xe máy:** xxYZxxxxx (8-9 ký tự)
 - xx: Mã tỉnh (2 số)
 - Y: Seri (1 chữ)
 - Z: Số/chữ
 - xxxxx: 4-5 số cuối
 - VD: 29B12345, 30M04019
- **Ô tô:** xxYxxxxx (7-8 ký tự)
 - VD: 51A1234, 30H12345

Retry Mechanism:

- Chỉ lưu kết quả đúng format (validated)
- Sai format → retry frame tiếp theo
- Đã có kết quả valid → dừng OCR cho xe đó

Môi trường thực nghiệm

Thành phần	Thông số
<i>Huấn luyện (Kaggle Cloud)</i>	
GPU	NVIDIA Tesla P100-PCIE (16GB)
CPU	4 Logical Cores
RAM	31.35 GB
<i>Inference (Local)</i>	
GPU	RTX 4060 / RTX 2050
Framework	PyTorch 2.0, CUDA 12.4

Bảng: Cấu hình môi trường thực nghiệm

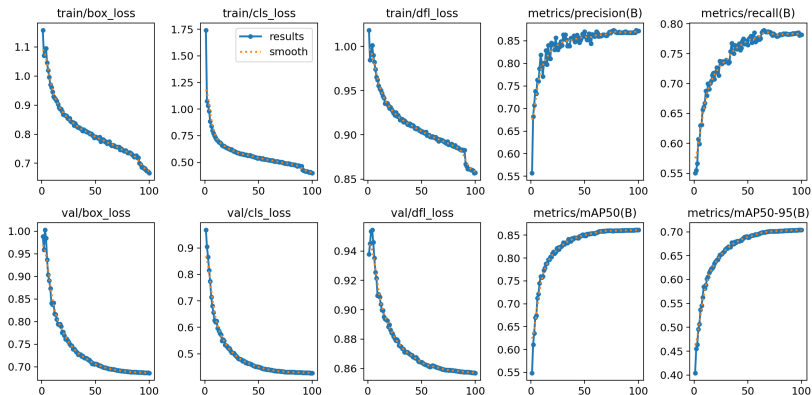
So sánh kết quả huấn luyện 3 mô hình

Mô hình	Precision	Recall	mAP@50	mAP@50-95
Model 1: Phát hiện xe (YOLOv8n, 5 lớp)	87.2%	78.1%	86.1%	70.4%
Model 2: Phát hiện biển (YOLOv8n, 1 lớp)	99.97%	99.86%	99.50%	81.90%
Mô hình tích hợp (YOLOv8n, 6 lớp)	84.7%	71.6%	78.5%	62.3%

Bảng: So sánh kết quả 3 mô hình Detection

- **Model 2 (Biển số):** Đạt kết quả tốt nhất nhờ bài toán đơn lớp
- **Model tích hợp:** Hiệu năng thấp hơn do class imbalance

Kết quả Model 1: Phát hiện phương tiện I



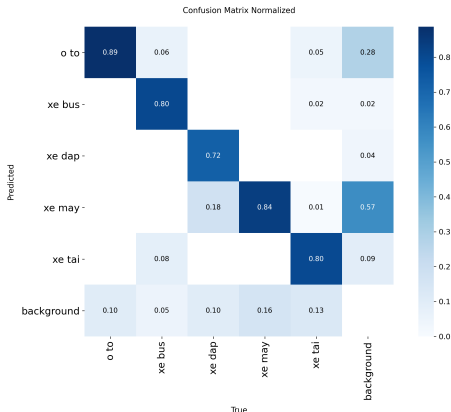
Hình: Đồ thị Loss và mAP - Model 1

Kết quả Model 1: Phát hiện phương tiện II

Kết quả chi tiết theo từng lớp:

Lớp	mAP@50	Precision	Recall
Ô tô	0.922	0.91	0.85
Xe buýt	0.897	0.88	0.80
Xe máy	0.872	0.85	0.79
Xe tải	0.832	0.84	0.75
Xe đạp	0.782	0.78	0.72

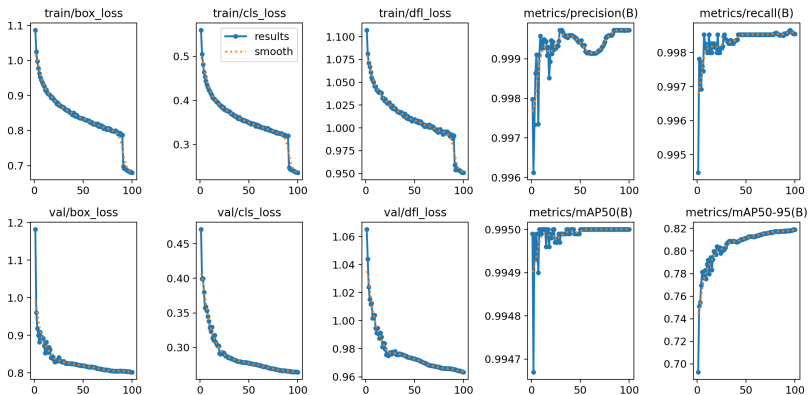
Kết quả Model 1: Phát hiện phương tiện III



Hình: Ma trận nhầm lẫn Model 1

Nhận xét: Nhầm lẫn chủ yếu giữa xe đạp và xe máy (18%)

Kết quả Model 2: Phát hiện biển số I



Hình: Đồ thị Loss và mAP - Model 2 (Plate Detection)

Kết quả Model 2: Phát hiện biến số II

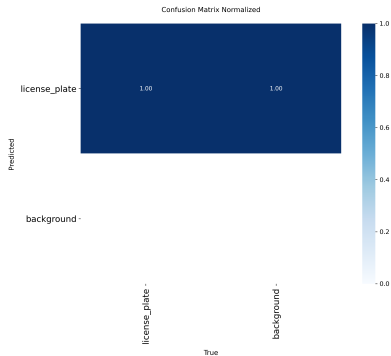
Kết quả xuất sắc

- **Precision:** 99.97%
- **Recall:** 99.86%
- **mAP@50:** 99.50%
- **mAP@50-95:** 81.90%

Lý do đạt kết quả cao:

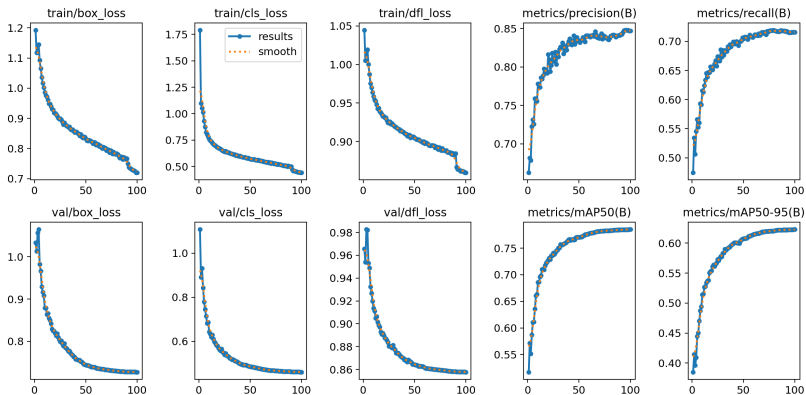
- Bài toán phát hiện 1 lớp (single-class)
- Pretrained weights chất lượng
- Dataset VNLP đa dạng, được gán nhãn cẩn thận

Kết quả Model 2: Phát hiện biển số III



Hình: Ma trận nhầm lẫn Model 2 - Gần như hoàn hảo

Kết quả Mô hình tích hợp 6 lớp I



Hình: Đồ thị Loss và mAP - Mô hình tích hợp

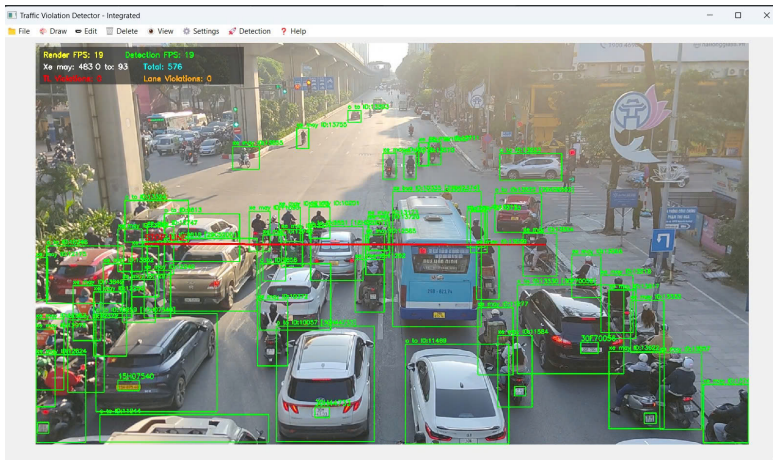
Kết quả Mô hình tích hợp 6 lớp II

Kết quả theo từng lớp:

Lớp	mAP@50	Precision	Recall
Ô tô	0.89	0.88	0.82
Xe buýt	0.86	0.85	0.78
Xe máy	0.84	0.83	0.76
Xe tải	0.81	0.82	0.73
Xe đạp	0.72	0.75	0.65
Biển số	0.59	0.72	0.56

Hạn chế: Lớp biển số có mAP thấp (0.59) do class imbalance và kích thước nhỏ

Kết quả Inference Pipeline 2 (One-Stage) I



Hình: Pipeline 2: Phát hiện đồng thời xe và biển số (1 model)

Kết quả Inference Pipeline 2 (One-Stage) II

Ưu điểm quan sát được:

- Phát hiện tốt các loại phương tiện với bbox chính xác
- Biển số được phát hiện khi có kích thước đủ lớn
- Mapping plate→xe tự động qua containment check (95% thành công)

Hạn chế:

- Biển số nhỏ/xa có thể bị bỏ sót
- Trade-off: Tốc độ nhanh nhưng mAP plate giảm

So sánh hiệu năng hai Pipeline ALPR

Tiêu chí	Pipeline 1 (Two-Stage)	Pipeline 2 (One-Stage)
FPS trung bình	5–10 FPS	15–20 FPS
mAP@50 (Plate)	99.5%	59%
Số model cần load	2	1
GPU Memory	2.5GB	1.2GB
Phát hiện biển nhỏ	Tốt	Trung bình
Tài nguyên tiêu thụ	Cao	Thấp

Bảng: So sánh hai phương pháp ALPR (theo báo cáo)

Khuyến nghị sử dụng

Pipeline 1 (Two-Stage) - Ưu tiên độ chính xác

- Yêu cầu độ chính xác cao cho biển số
- Camera có độ phân giải tốt
- Biển số nhỏ hoặc ở xa

Pipeline 2 (One-Stage) - Ưu tiên tốc độ

- Yêu cầu xử lý real-time
- Tài nguyên phần cứng hạn chế
- Biển số đủ lớn trong khung hình

Đánh giá ByteTrack Tracking

Kết quả thực tế

- **ID Switch Rate:** $< 5\%$ trên video thử nghiệm
- **Overhead:** 2ms mỗi frame
- **Tính năng:** Kalman Filter + Hungarian Matching

Quan sát thực tế

- **Hoạt động tốt:** Xe di chuyển bình thường, mật độ cao
- **ID switch xảy ra khi:**
 - Xe bị che khuất > 2 giây
 - Thay đổi tốc độ đột ngột
 - Xe quá gần nhau (occlusion)

Cải thiện tiềm năng: Kết hợp Re-ID features

Hiệu quả Format Validation

- Loại bỏ kết quả nhiễu ngay từ đầu
- Chỉ hiển thị biển số đúng format VN
- Tận dụng tracking để retry tự động

Các trường hợp khó:

- 1 Biển số bản/mờ: CLAHE giúp cải thiện nhưng hạn chế
- 2 Góc nghiêng lớn: Perspective distortion giảm độ chính xác
- 3 Ban đêm: Phản sáng từ đèn flash làm mất chi tiết
- 4 Biển nhỏ/xa: Độ phân giải thấp

Tính năng chính:

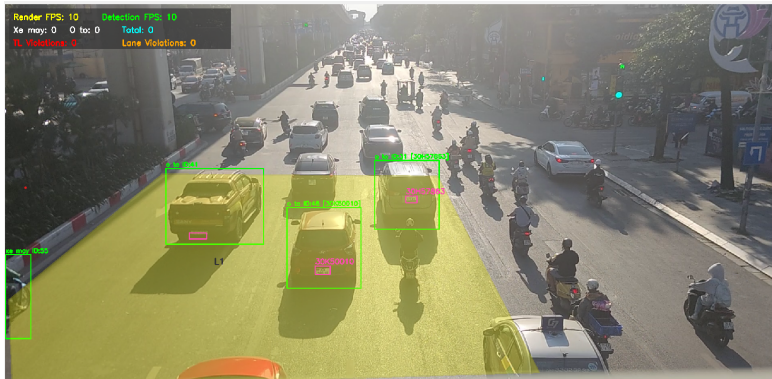
- Chọn video/camera đầu vào
- Cấu hình ROI (Stopline, Lanes)
- Chọn Pipeline ALPR
- Hiển thị real-time
- Xuất báo cáo vi phạm

Framework: PyQt5 + OpenCV

Thành phần UI:

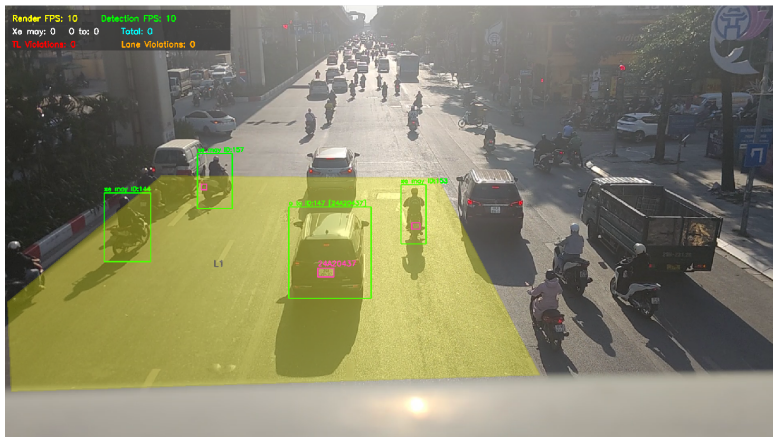
- Control Panel (bên trái)
- Video Display (giữa)
- Info Panel (bên phải)
- Status Bar (dưới)

Kết quả Inference thực tế I



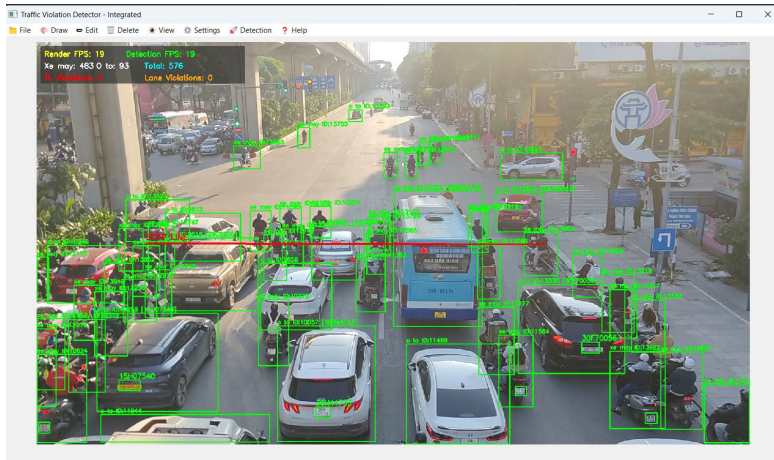
Hình: Pipeline 1: Phát hiện biển số chính xác

Kết quả Inference thực tế II



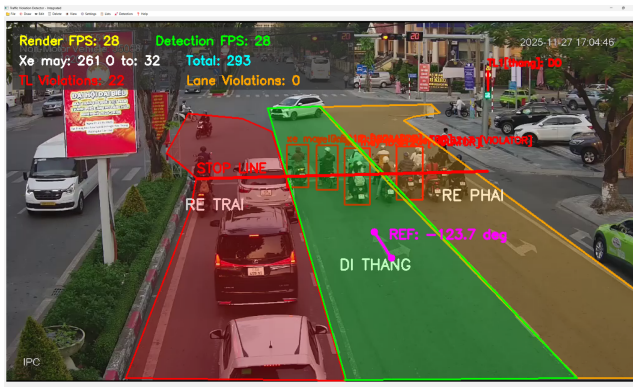
Hình: Pipeline 1: OCR nhận dạng biển số Việt Nam

Kết quả Inference thực tế III



Hình: Pipeline 2: Phát hiện đồng thời xe và biển số

Kết quả phát hiện vi phạm I



Hình: Phát hiện xe vượt đèn đỏ (Bbox đỏ = vi phạm)

Kết quả phát hiện vi phạm II



Hình: Phân loại xe tuân thủ (Bbox xanh) và vi phạm (Bbox đỏ)

Đánh giá hệ thống phát hiện vi phạm

Kịch bản 1: Vượt đèn đỏ

- Logic so sánh tọa độ xe với vạch dừng ảo hoạt động chính xác
- Kết hợp với trạng thái đèn (HSV) để đưa ra quyết định
- Hỗ trợ 4 loại đèn: Tròn, Rẽ trái, Rẽ phải, Đi thẳng

Kịch bản 2: Đi sai làn

- Thuật toán Point-in-Polygon xác định xe thuộc làn nào
- So sánh loại xe với danh sách cho phép của làn
- Hoạt động tốt ngay cả với góc camera thấp

Thách thức và Hạn chế

Thách thức đã giải quyết:

- Mật độ giao thông cao
- Điều kiện ánh sáng thay đổi
- Đa dạng loại biển số VN
- Góc camera nghiêng

Hạn chế còn tồn tại:

- Biển số bẩn/mờ
- Biển số nhỏ/xa (Pipeline 2)
- Tốc độ giảm khi nhiều xe
- Cấu hình ROI thủ công

Tổng kết

Kết quả đạt được (theo báo cáo)

- **Phát hiện phương tiện:** $mAP@50 = 86.1\%$
- **Phát hiện biển số:** $mAP@50 = 99.5\%$ (Pipeline 1)
- **Mô hình tích hợp:** $mAP@50 = 78.5\%$ (all classes)
- **Tracking:** ID switches $< 5\%$
- **Vi phạm:** False positive $< 2\%$
- **Tốc độ:** 5–20 FPS tùy pipeline

Hướng phát triển

- Tích hợp auto-retry khi OCR confidence thấp
- Hỗ trợ cross-camera tracking
- Tự động phát hiện ROI bằng Lane Detection
- Triển khai trên edge devices (Jetson)

① So sánh 2 Pipeline ALPR:

- Two-Stage: mAP 99.5%, 5–10 FPS
- One-Stage: mAP 78.5%, 15–20 FPS

② Huấn luyện 3 mô hình:

- Model 1: YOLOv8n (5 lớp xe) - mAP 86.1%
- Model 2: YOLOv8n (1 lớp plate) - mAP 99.5%
- Model tích hợp: YOLOv8n (6 lớp) - mAP 78.5%

③ Hệ thống phát hiện vi phạm:

- Vượt đèn đỏ (4 loại đèn)
- Đi sai làn (Point-in-Polygon)
- Direction Fusion (Trajectory + ROI)

④ Ứng dụng: PyQt5, cấu hình JSON, xuất báo cáo

Q&A

Mời thầy cô và các bạn đặt câu hỏi

**Cảm ơn thầy cô và các bạn
đã lắng nghe!**